

PROJECT 3 — AI INVESTMENT AGENTS

Value Investing Challenge • RA3

EVERY PREDICTION MUST BE RECORDED BEFORE WE KNOW THE OUTCOME.

Una predicción no es "esta acción va a subir". Una predicción es: *"dada la información disponible hoy, mi sistema estima un 67% de probabilidad de que esta acción alcance al menos +10% en los próximos 3 meses"*. Un LLM puede escribir esa frase con total fluidez y estar completamente descalibrado. En RA3 el reto no es generar texto convincente: es que el número sea correcto.

1. Introducción del proyecto

Ya tenéis un Data Lake (RA1), un Data Warehouse con `STOCK_FEATURES` point-in-time y modelos de ML calibrados (RA2). Ahora construís agentes de IA que enriquecen ese sistema: agentes especializados con acceso a herramientas sobre el warehouse, capaces de razonar sobre datos estructurados e información externa, coordinados por un comité de inversión que emite la predicción final.

Seguís sin construir la web. La plataforma **Value Investing Challenge** consulta vuestro endpoint cada ciclo semanal, congela la respuesta de forma inmutable y os rankea. La tarea es idéntica: **Top 3 tickers por horizonte** para `1W`, `1M`, `3M`, `6M`, **máximo 12 predicciones por ciclo**. Una predicción es **hit** cuando `realized_return >= 0.10`.

En RA3 vuestro modelo se registra como `v3`. `v1` y `v2` permanecen intactos. El curso entero converge en una pregunta:

¿Los agentes realmente mejoraron las predicciones?

Y en la advertencia que la acompaña: **la complejidad no se premia sin resultados**. Un enjambre de siete agentes que puntúa por debajo de vuestro propio XGBoost de RA2 es un resultado válido — siempre que lo digáis, lo midáis y lo analicéis. Ocultarlo, no.

2. Objetivos de aprendizaje

- Diseñar un sistema multi-agente con responsabilidades separadas y justificar la arquitectura elegida.
- Implementar **tools** que consulten el warehouse, de modo que el agente lea números reales en lugar de inventarlos.

- Forzar **structured output** (JSON schema / function calling) para producir un payload válido de forma determinista.
- Orquestar agentes: secuencial, paralelo, jerárquico, con agregación de resultados.
- Aplicar prompting y context engineering: qué entra en el contexto, en qué orden y con qué presupuesto de tokens.
- Usar RAG cuando aporte (transcripts, filings, noticias) y reconocer cuándo no aporta.
- Producir probabilidades **calibradas** con un LLM, que es lo contrario de su comportamiento por defecto.
- Comparar `v3` contra `v2` de forma like-for-like y sacar una conclusión defendible.
- Controlar coste y reproducibilidad de un pipeline no determinista con una fecha límite semanal.

3. Arquitectura



4. Agentes candidatos

No estáis obligados a implementar exactamente cinco. Elegís la arquitectura y la justificáis. Tres agentes bien contruidos con tools reales valen más que seis que se pasan texto genérico.

AGENTE	ÁMBITO	TOOLS TÍPICAS SOBRE EL WAREHOUSE
Fundamental Agent	Valoración, crecimiento, rentabilidad, FCF, ROIC, deuda, calidad de la compañía	<code>get_valuation_metrics</code> , <code>get_profitability</code> , <code>get_leverage</code>
Earnings Agent	Earnings, sorpresas, guidance, transcripts de earnings calls, sentimiento de la dirección, cambios respecto a llamadas anteriores	<code>get_earnings_history</code> , <code>get_surprise</code> , <code>search_transcript</code>
Competitor Agent	Valoración, crecimiento, márgenes y ROIC frente a comparables; posicionamiento sectorial	<code>get_peers</code> , <code>get_sector_relative_metrics</code>

AGENTE	ÁMBITO	TOOLS TÍPICAS SOBRE EL WAREHOUSE
Market Agent	Momentum, volatilidad, volumen, momentum sectorial, régimen de mercado	<code>get_momentum</code> , <code>get_volatility</code> , <code>get_market_regime</code>
News Agent	Noticias de compañía y sector, revisiones de analistas, eventos relevantes	<code>search_news</code> , <code>get_analyst_revisions</code>
Investment Committee Agent	Consume la salida de los demás y emite, por ticker: probabilidades 1W/1M/3M/6M, <code>expected_return</code> , <code>investment_thesis</code> , <code>risks</code> , <code>confidence</code>	<code>get_ml_prior</code> (la probabilidad de vuestro modelo de RA2)

5. Qué se evalúa de verdad: no es "llamar a un LLM"

Seis capacidades concretas:

1. Tools sobre el warehouse. Un agente que ejecuta `SELECT roic, pe_ratio FROM stock_features WHERE ticker='NVDA' AND snapshot_date='2026-09-14'` es incomparablemente más útil que uno que "recuerda" que NVDA tiene buen ROIC. El LLM no debe generar números financieros: debe pedirlos. Todo dato cuantitativo del razonamiento tiene que ser trazable a una fila del warehouse.

Corolario point-in-time, heredado de RA2: las tools deben aceptar `snapshot_date` y filtrar por él. Una tool que devuelve "los últimos datos disponibles" mete look-ahead bias cuando reconstruís histórico.

2. Structured output. El contrato exige JSON válido. Texto libre de un LLM es un modo de fallo garantizado: un payload inválido se rechaza por completo y esa semana no puntuáis. Usad JSON schema estricto o function calling, validad la salida contra el esquema antes de servirla, y tened un camino de reintento y un fallback (por ejemplo, servir la predicción de `v2`) si el agente devuelve algo inválido.

3. Orquestación. Decidid y justificad: agentes en paralelo con agregación posterior (rápido, sin contaminación entre agentes), o secuencial con contexto acumulado (más rico, más caro, con riesgo de anclaje del primer agente).

4. Agentes especializados. Un prompt por rol, con su propio conjunto de tools. Un único agente con veinte tools y un prompt de tres páginas rinde peor y es imposible de depurar.

5. Razonamiento sobre datos estructurados e información externa. El valor añadido frente a RA2 es procesar lo que un XGBoost no puede: transcripts, guidance, contexto cualitativo de un sector.

6. Comparación ML vs agentes. Es un entregable, no una reflexión final.

6. Probabilidades calibradas con un LLM

Los LLM son notoriamente sobreconfiados y gravitan hacia números redondos: 70%, 80%, 90%. Sobre un horizonte de 1W con tasa base del 9%, un agente que declara 0.8 no está optimista, está produciendo un Brier Skill Score negativo.

Tasas base reales medidas en el dataset de referencia:

HORIZONTE	HIT RATE OBSERVADO
1W	≈ 9%
1M	≈ 35%
3M	≈ 43%
6M	≈ 42%

Quien declare 70% de probabilidad en un pick de 1W tiene garantizado un Brier score terrible. Es una de las lecciones centrales del curso.

Grounding sobre el prior del ML

Enfoque recomendado: no pidáis al agente una probabilidad desde cero. Dadle la probabilidad calibrada de vuestro modelo de RA2 como **prior** y dejadle ajustarla dentro de un rango acotado, con justificación obligatoria.

```
Prior del modelo ML para NVDA / 3M: 0.38
Tasa base de la cohorte para 3M: 0.43
Ajuste permitido: ±0.10 en términos absolutos.
Debes justificar el ajuste con evidencia de las tools.
Si no tienes evidencia que contradiga al prior, devuelve el prior.
```

Esto tiene dos ventajas medibles. Primera: acota el daño de la sobreconfianza, porque el punto de partida ya está calibrado. Segunda, y más importante académicamente: **aísla la contribución del agente**. Podéis medir el Brier score con el prior puro y con el prior ajustado y responder si el ajuste mejoró algo. Si el ajuste medio empeora el Brier, la conclusión es que el agente añade ruido, y eso es un hallazgo legítimo.

Otras medidas prácticas: prohibid explícitamente los múltiplos de 0.05 para forzar salidas no ancladas en números redondos; incluid la tasa base del horizonte en el contexto; y aplicad una recalibración posterior sobre las salidas del agente con el mismo aparato de RA2 (Platt scaling) si tenéis suficientes observaciones.

7. Coste y reproducibilidad

Un pipeline no determinista con fecha límite semanal es un riesgo operativo. La plataforma consulta vuestro endpoint en su ventana; si en ese momento el agente entra en bucle, agota su presupuesto de tokens o devuelve JSON malformado, esa semana no puntuáis.

RIESGO	MITIGACIÓN CONCRETA
Coste por ciclo	Modelos pequeños para agentes especializados, modelo grande solo para el comité
Llamadas repetidas en desarrollo	Caché de llamadas LLM por hash de (modelo, prompt, params). Reduce el coste de iterar a casi cero
No determinismo	<code>temperature=0</code> y <code>seed</code> fijo donde el proveedor lo soporte. No garantiza reproducibilidad exacta, pero reduce la varianza
Latencia en la ventana de recogida	Precalcular las predicciones en un job programado y que el endpoint sirva el resultado almacenado. El endpoint nunca debe invocar agentes en el momento de la petición
Salida inválida	Validación contra esquema + reintento + fallback a <code>v2</code>
Sin presupuesto disponible	Modelos con free tier o modelos locales pequeños vía Ollama; degradar a <code>v2</code> antes que no entregar

Registrad por ciclo: número de llamadas, tokens, coste y latencia. Es un entregable y además la única forma de argumentar si el sistema es viable.

8. Requisitos

Obligatorio: al menos tres agentes especializados con tools reales sobre el warehouse, un agente comité que agrega, structured output validado contra esquema, precálculo de predicciones, y la comparación `v2` vs `v3`.

Requisitos mínimos (para aprobar)

- Mínimo 3 agentes con roles distintos y tools funcionales que consulten el warehouse.
- Todo dato cuantitativo del razonamiento trazable a una fila del warehouse.
- Structured output con validación contra esquema y manejo de fallos.
- Comité que emite probabilidades para los 4 horizontes, `expected_return`, `investment_thesis` y `risks`.
- Probabilidades ancladas en el prior del ML o en la tasa base del horizonte, con justificación.
- Endpoint `GET /api/predictions` sirviendo `model_version: "v3"` con payload válido.
- Informe comparativo `v2` vs `v3` like-for-like con conclusión explícita.
- Registro de coste y latencia por ciclo.

Opcional / bonus

- RAG sobre transcripts de earnings calls o filings, con citación de la fuente en la tesis.
- Detección de cambios entre earnings calls consecutivas.
- Debate entre agentes con posiciones contrarias y resolución razonada.
- Medición de la aportación marginal de cada agente (ablación: quitar uno y medir el Brier).

- Recalibración de las salidas del agente con Platt scaling.
- Evaluación automática de la calidad del razonamiento con LLM-as-judge.
- Modelo local para reducir coste a cero, con la degradación de calidad medida.

9. Tecnologías y conceptos

LLMs, structured outputs, tools, agentes, sistemas multi-agente, prompting, context engineering, RAG cuando sea apropiado. No se impone framework: podéis usar orquestación propia con llamadas directas a la API, o un framework de agentes. Un bucle de 200 líneas escrito por vosotros es más fácil de depurar y de explicar que un framework que no dominéis.

10. Entregables

1. Repositorio con el código de los agentes, las tools y la orquestación.
2. Diagrama de la arquitectura de agentes y documento justificando por qué esa y no otra.
3. Prompts versionados en el repositorio, no incrustados en cadenas dispersas.
4. Esquema JSON de la salida y el código de validación.
5. Trazas de ejecución de al menos un ciclo completo: qué tools se llamaron, con qué argumentos, qué devolvieron y cómo se llegó a la probabilidad final.
6. Informe de calibración: probabilidades del prior de ML frente a las ajustadas por el agente, con Brier de ambas.
7. Informe de coste y latencia por ciclo.
8. Informe `v2` vs `v3`, y si es posible `v1` vs `v2` vs `v3`.
9. Endpoint desplegado sirviendo `v3`.

11. Rúbrica de evaluación

CRITERIO	PESO	INSUFICIENTE	CORRECTO	EXCELENTE
Arquitectura de agentes	15%	Una sola llamada a un LLM	3+ agentes con roles separados	Arquitectura justificada, orquestación razonada, aportación marginal medida
Tools sobre el warehouse	20%	El LLM inventa los números	Tools funcionales que leen el warehouse	Tools con <code>snapshot_date</code> , manejo de errores, y trazabilidad completa dato→fila
Structured output y robustez	15%	Texto libre parseado con regex	JSON schema validado	Validación, reintento, fallback a <code>v2</code> y cero payloads rechazados

CRITERIO	PESO	INSUFICIENTE	CORRECTO	EXCELENTE
Calibración de probabilidades	20%	Números redondos sobreconfiados	Anclaje en prior de ML o tasa base	Ajuste acotado, medido y comparado contra el prior con Brier
Comparación v2 vs v3	15%	Sin comparación o cherry picking	Comparación numérica like-for-like	Análisis de causas, ablaciones, conclusión defendible aunque sea negativa
Coste y reproducibilidad	10%	Agentes invocados en la petición HTTP	Predicciones precalculadas, coste registrado	Caché de llamadas, determinismo controlado, plan de degradación
Documentación y trazas	5%	Sin trazas	Trazas de un ciclo completo	Trazas legibles que permiten auditar cualquier predicción

12. Cherry picking y complejidad sin resultados

Dos errores metodológicos que en RA3 pesan más que en cualquier otro RA.

Cherry picking

Reportar solo la semana, el horizonte o el ticker donde los agentes ganaron. Con 12 predicciones por ciclo y 4 horizontes, siempre hay algún subconjunto donde **v3** bate a **v2**. Encontrarlo no es un resultado: es aritmética.

La comparación debe ser **like-for-like**:

- Mismos ciclos. Comparar **v3** de un mes con **v2** de otro mes mide el mercado, no vuestro sistema.
- Mismos horizontes. El Brier de 1W y el de 6M no son comparables: las tasas base son 9% y 42%.
- Mismo universo de tickers.
- Todos los horizontes reportados, no solo los favorables.
- Muestra declarada. Con 12 predicciones resueltas, una diferencia de hit rate de 8 puntos es una predicción. **Small sample size**: si no podéis descartar el azar, decidlo.

Formato mínimo del informe comparativo, por horizonte:

HORIZONTE	N RESUELTAS	HIT RATE V2	HIT RATE V3	BRIER V2	BRIER V3	BSS V2	BSS V3
1W							
1M							
3M							
6M							

Y una fila agregada. Sin celdas vacías por conveniencia.

Complejidad sin resultados

La complejidad no se premia sin resultados. Siete agentes, RAG, debate multi-turno y un grafo de orquestación elegante no valen nada si el Brier score empeora respecto a RA2.

Si vuestro sistema de agentes rinde por debajo de vuestro propio XGBoost, el trabajo consiste en decirlo, medirlo y analizar por qué. Hipótesis a examinar con datos:

- El agente sobrescribió priors calibrados con juicio no calibrado. Comprobable: Brier del prior puro vs prior ajustado.
- Las tools devolvían datos insuficientes y el agente rellenó los huecos con conocimiento genérico.
- El sesgo hacia números redondos dominó la señal.
- Los agentes coincidían entre sí y no añadían información independiente. Comprobable por correlación entre sus salidas.
- La muestra es demasiado pequeña para distinguir las dos versiones.

Ese análisis honesto puntúa más que esconder el resultado. Un informe que concluye "los agentes no mejoraron el Brier en 3M y esta es la evidencia de por qué" es un trabajo mejor que uno que presenta la única semana favorable como prueba de éxito.

13. Errores comunes

- **El LLM genera cifras financieras** en lugar de pedir las a una tool. Alucinación con formato de informe.
- **Endpoint que invoca agentes en la petición.** Timeout en la ventana de recogida y semana perdida.
- **JSON malformado** sin validación previa. Rechazo total del payload.
- **generated_at** sin offset, generado por el LLM en lugar de por código. Rechazo total.
- **Probabilidades en múltiplos de 0.1** para los 12 picks. Señal inequívoca de que nadie calibró nada.
- **Sobreconfianza en 1W.** La tasa base es 9%.
- **Un agente con veinte tools** y un prompt gigante. Imposible de depurar y peor rendimiento.
- **Tools sin snapshot_date** que devuelven datos posteriores a la fecha de la predicción.
- **Prompts no versionados.** Los resultados dejan de ser reproducibles.
- **Cherry picking** del horizonte favorable.
- **Sin fallback.** Cuando el proveedor falla, no hay nada que servir.
- **Usar el dataset sintético de data/mock/** para generar predicciones enviadas.

14. El dataset de referencia

En `data/mock/` del repositorio de la plataforma, en CSV y JSON: `companies.csv`, `market_data.csv`, `fundamentals.csv`, `earnings.csv`, `competitors.csv`, `news_signals.csv`, `macro.csv`, `stock_features.csv`, `predictions.csv`, `prediction_results.csv`, `cycles.csv`, `students.csv`.

Universo: AAPL, MSFT, NVDA, META, GOOGL, AMZN, TSLA, JPM, V, NFLX. Sintético pero internamente consistente: un mayor `revenue_growth` produce realmente un `growth_score` mayor. En RA3 es especialmente útil para desarrollar y probar las tools sin consumir cuota de API ni presupuesto de tokens.

No enviéis predicciones basadas en este dataset sintético. Es solo para desarrollo.

15. Cómo se os puntúa

Métricas por alumno: Resolved predictions, Hit Rate, Average/Median/Best/Worst Return, Average Alpha (contra un benchmark equiponderado), Average Probability, Brier Score, Brier Skill Score, Calibration y Consistency.

- **Brier score:** media de $(p - y)^2$ con $y=1$ si hit. Más bajo, mejor.
- **Brier Skill Score:** $1 - \text{brier} / \text{brier_reference}$, con la referencia siempre en la tasa base de la cohorte para ese horizonte. Positivo = vuestras probabilidades aportan información. **Negativo = son activamente engañosas**, que es donde acaba el exceso de confianza de un LLM.
- **Calibration:** 10 bins de probabilidad, comparando probabilidad declarada contra tasa de acierto observada.
- **Leaderboard score (0–100)** = 30% hit rate + 25% calibration + 25% relative return + 10% consistency + 10% sample reliability. Cada componente **relativo a la cohorte**, no contra una constante absoluta.
- Mínimo **12 predicciones resueltas** para aparecer en el ranking principal.

Nótese que calibración pesa un 25%: en un proyecto de agentes es probablemente el componente donde más fácil es perder puntos y donde el grounding sobre el prior de ML rinde más.

16. Formato de entrega

- Repositorio Git con historial de commits.
- README con: arquitectura, cómo ejecutar el pipeline de agentes, variables de entorno necesarias (sin credenciales en el repositorio), cómo servir el endpoint.
- Prompts en ficheros versionados.
- Trazas de ejecución de un ciclo, en el repositorio.
- Informe `v2` vs `v3` en Markdown, con las tablas por horizonte.

- Endpoint accesible por HTTPS y respondiendo durante la ventana de recogida.

17. Conexión con el Value Investing Challenge

La plataforma consulta vuestro endpoint una vez por ciclo, guarda la respuesta de forma inmutable y no la modifica jamás. Cuando el horizonte vence, calcula el retorno realizado y resuelve la predicción. **Toda predicción queda registrada antes de conocer el resultado.** Con agentes esto es una restricción de ingeniería concreta: la respuesta que sirváis en la ventana de recogida es la que se juzga, sin segunda oportunidad y sin edición posterior.

Contrato del alumno

GET /api/predictions debe devolver:

```
{
  "student_id": "student-01",
  "model_version": "v1",
  "model_name": "Weighted Value Score",
  "generated_at": "2026-09-14T12:00:00Z",
  "predictions": [
    {
      "ticker": "META",
      "horizon": "1W",
      "rank": 1,
      "probability": 0.71,
      "expected_return": 0.14,
      "target_price": 712.40,
      "investment_thesis": "optional free text",
      "risks": "optional free text"
    }
  ]
}
```

Reglas de validación (Zod) aplicadas por la plataforma: `horizon` $\in \{1W, 1M, 3M, 6M\}$; `rank` $\in 1..3$ y único por horizonte; `probability` $\in [0, 1]$; `expected_return` $\in [-1, 10]$; `generated_at` debe ser un datetime ISO-8601 **con offset**; ticker en mayúsculas; sin tickers duplicados dentro del mismo horizonte; máximo 12 predicciones. **Un payload que falla la validación se rechaza por completo y esa semana el alumno no puntúa.** La guía completa está en `docs/STUDENT_INTEGRATION.md`.

En RA3 debéis enviar `model_version: "v3"`. El ejemplo de arriba es el contrato literal, que muestra `v1`; cambiad ese campo. Los campos `investment_thesis` y `risks` son opcionales en el contrato, pero en RA3 son la salida natural del comité: usadlos.

La plataforma conserva `v1`, `v2` y `v3` por separado. El curso se cierra con dos respuestas empíricas: **¿el ML batió al scoring manual?** y **¿los agentes batieron al ML?** Vuestro trabajo es producir esas dos respuestas con evidencia, sea la que sea.

Every prediction must be recorded before we know the outcome. • PROJECT 3 — AI INVESTMENT AGENTS